such as SHA-384 and SHA-512. There is also now a newer SHA-3 standard (selected through an open international competition), with variants named SHA3-256, SHA3-384, and so on, but SHA-2 is still considered secure and is widely implemented.

CSRF tokens, CRIME, and BREACH

CSRF tokens and session cookies are protected during transmission between the client and server by TLS, which encrypts the connection and hides these tokens from observers. Although there have been many weaknesses in TLS over the years, most of these have been fixed. However, there are some attacks related to the use of compression that have never been widely fixed, and these attacks can allow CSRF tokens and cookies to be stolen. The first such attack was known as CRIME (https://en.wikipedia.org/wiki/CRIME) and exploited a vulnerability in the compression built into TLS itself. This compression was subsequently disabled, but another variant of the attack called BREACH (http://breachattack.com) exploited the compression in HTTP instead. HTTP compression is just too useful to turn off, so this attack has never been fully mitigated.

Both of these attacks, and subsequent variations, depend on the same token value being used for many requests and responses. A countermeasure is therefore to ensure that the token changes on every request, for example by masking it with a random value. The random value can be XORed with the CSRF token and sent alongside it to the server, which can then undo the XOR operation to recover the original token. This ensures that the token is different on every request. Hanno Böck has suggested a scheme (https://blog.hboeck.de/archives/900-Generating-CRIME-safe-CSRF-Tokens.html) in which the random value is also hashed with a string representing the API operation, ensuring that even if a token is stolen it can only be used for the same type of requests. In chapter 9 you'll develop capability URIs which also use unique tokens for each request.

4.4.3 Double-submit cookies for the Natter API

To protect the Natter API, you will implement hash-based double-submit cookies as described in the last section. First, you should update the `CookieTokenStore` create method to